Optimal LTL$_f$ Synthesis

Yujian Cao, Sven Schewe, Qiyi Tang, Shufang Zhu

University of Liverpool

{yujian.cao, sven.schewe, qiyi.tang, shufang.zhu}@liverpool.ac.uk

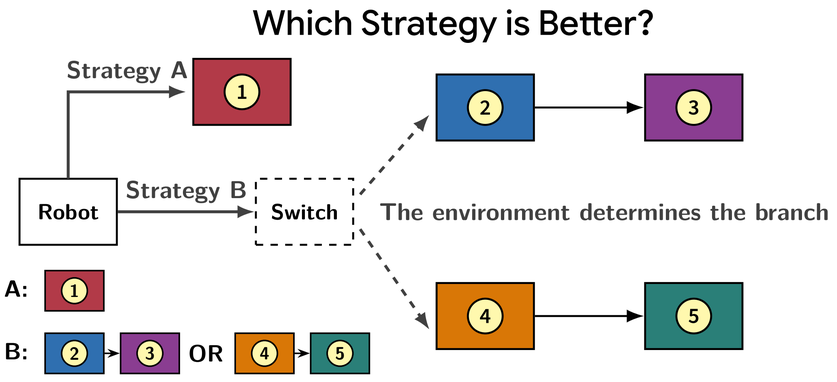
Full Version TCS@Liverpool

IJCAI ECAI BREMEN 2026

Motivation: When LTL$_f$ objectives are not jointly realisable

The Limitations of Standard LTL$_f$ Synthesis: An agent must satisfy a **single LTL$_f$ formula** while interacting with a **fully adversarial environment**; users may struggle to capture all desired objectives in a single specification.

Optimal LTL$_f$ Synthesis: LTL$_f$ synthesis with **multiple objectives**, especially for the case that they are not all jointly realisable; an optimal strategy should *"satisfy as many as possible"*.



Optimal LTL$_f$ Synthesis: Max-Guarantee & Max-Observation

Given a set of LTL$_f$ formulas $\Phi = (\varphi_1, \varphi_2, \cdots, \varphi_n)$, we want to *satisfy as many as possible*...

Max-Guarantee Synthesis: Commit to a **specific** maximal (or maximally valued) subset of objectives in advance.

- **First** select a subset $\Psi \subseteq \Phi$ that we promise to realise, and **then** synthesise the strategy with the goal of realising Ψ.

Max-Observation Synthesis: Maximise the minimum number (or value) of objectives realised over all executions.

- Find a strategy that may realise **different subsets** of Φ, depending on the environment's behaviour.

Specific subset: 1

Different subsets: 2 3 OR 4 5

Solution: Shared Construction

Step 1: Automata Construction. For every LTL$_f$ formula $\varphi_i \in \Phi$, we build its corresponding DFA $\mathcal{A}^i = (2^{\mathcal{X}\cup\mathcal{Y}}, Q^i, q_0^i, \delta^i, F^i)$, where $F^i \subseteq Q^i$ is the set of accepting states for φ_i.

Step 2: Game Arena Construction. We build the product automaton $\mathcal{A} = \bigotimes_{\varphi_i\in\Phi} \mathcal{A}^i$, such that $\mathcal{A} = (2^{\mathcal{X}\cup\mathcal{Y}}, Q, q_0, \delta, \emptyset)$, where $Q = \prod_{\varphi_i\in\Phi} Q^i$ is the product state space, $q_0 = (q_0^i)_{\varphi_i\in\Phi}$ is the initial state, δ is the product transition function induced by $\{\delta^i\}_{\varphi_i\in\Phi}$.

For each $\varphi_i \in \Phi$, we define $F_\otimes^i = \{(q_1, \dots, q_n) \in Q \mid q_i \in F^i\}$ as the set of states in the product automaton where φ_i is among the satisfied specifications.

Step 3: Target Construction

Max-Guarantee

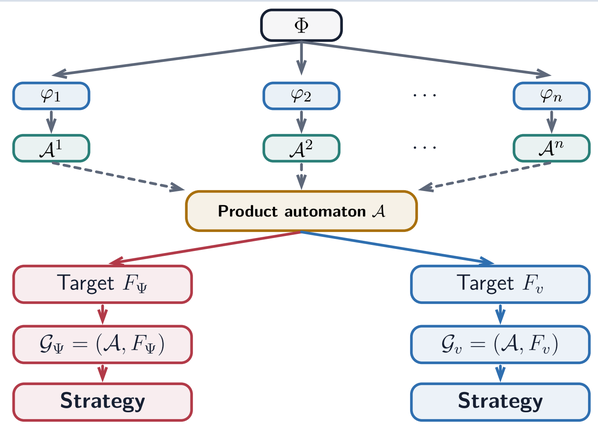
For each subset $\Psi \subseteq \Phi$, define $F_\Psi = \bigcap_{\varphi_i\in\Psi} F_\otimes^i$.

Step 4: Game Solving. Solve $\mathcal{G}_\Psi = (\mathcal{A}, F_\Psi)$ in non-increasing $\sum_{\varphi\in\Psi} G(\varphi)$. For the first Ψ with $q_0 \in \mathsf{Win}(\mathcal{G}_\Psi)$, return a winning strategy of $\mathcal{G}_\Psi$.

Max-Observation

For each value v taken by $\sum_{\varphi\in\Psi} V(\varphi)$, define $F_v = \bigcup\{F_\Psi \mid \Psi \subseteq \Phi,\ \sum_{\varphi\in\Psi} V(\varphi) \geq v\}$.

Step 4: Game Solving. Solve $\mathcal{G}_v = (\mathcal{A}, F_v)$ in decreasing v. Let v^* be the first v with $q_0 \in \mathsf{Win}(\mathcal{G}_v)$; return a winning strategy of $\mathcal{G}_{v^*}$.



Incremental Max-Observation Synthesis: Optimal for Every History

Extending the Basic Algorithm. After solving each DFA game $\mathcal{G}_v$, we record for every game state q the maximal value v_q such that $q \in \mathsf{Win}(\mathcal{G}_{v_q})$, together with the corresponding local strategy $\sigma_q \in 2^{\mathcal{Y}}$.

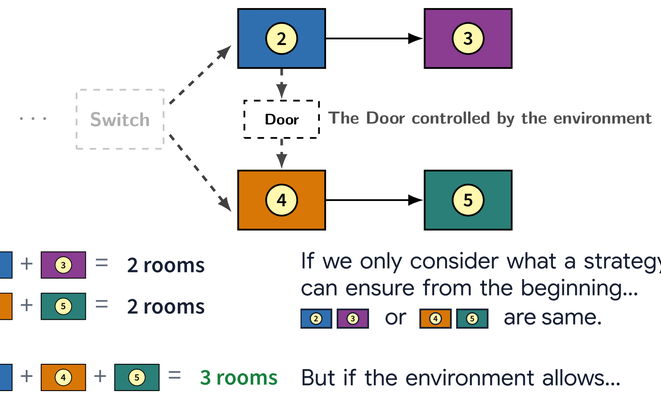
Combining these local strategies yields a global strategy σ. Whenever a game state q of the product automaton $\mathcal{A}$ is reached, σ follows the corresponding local strategy σ_q, thus achieving the maximal observed value v_q when q is treated as the initial state of the game. The resulting σ is an incremental observationally optimal strategy.

Improved Algorithm. The basic extension may consider each game state and state-action pair multiple times. At iteration k, the target T_k includes the previously computed winning region W_{k-1} and the new targets $F_{v_k}^=$. Let $\mathcal{G}_k = (\mathcal{A}, T_k)$.

$$T_k = W_{k-1} \cup F_{v_k}^=, \qquad W_k = \mathsf{Win}(\mathcal{G}_k) = \mathsf{Win}(\mathcal{G}_{v_k}).$$

The target sets grow monotonically as the value decreases. For any history h, if the maximal achievable value has not already been observed, the reached state q_h must be winning for that value.

We may be in a position to visit more rooms...



If we only consider what a strategy can ensure from the beginning...
2 3 or 4 5 are same.

But if the environment allows...

Experimental Evaluation

Implementation. We implemented **OptLtlfSynt**, using **Spot** for LTL$_f$-to-DFA translation and **CUDD** 3.0.0 for symbolic synthesis.

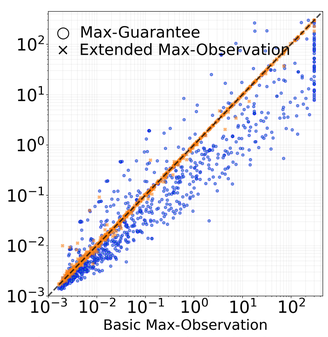
Benchmarks. We evaluate our approach on benchmarks taken from the LTL$_f$ synthesis track of SyntCOMP. All approaches are evaluated on **1,145** benchmarks that are **unrealisable and conjunction-decomposable**. Specifications are given as conjunctions of multiple individual LTL$_f$ formulas. The number of conjuncts per instance ranges from 2 to 50.

Approach	Solved	/ 1,145
Max-Guarantee	1,060	92.6%
Basic Max-Observation	1,005	87.8%
Extended Max-Observation	1,006	87.9%
Improved Max-Observation	1,008	88.0%

SyntCOMP: syntcomp.org Spot: spot.lre.epita.fr

Max-Observation can be strictly better than Max-Guarantee.
Max-Observation achieves a strictly larger ensured value on **16** instances.

Extending basic Max-Observation to Incremental Max-Observation has minor overhead.



Runtime: Basic Max-Observation vs. Max-Guarantee and Extended Max-Observation

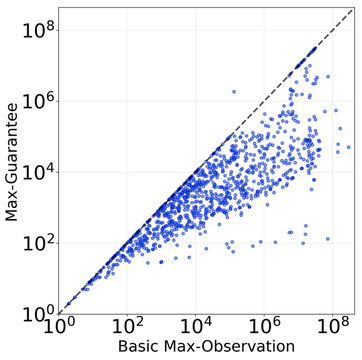
Max-Guarantee is obviously faster than Max-Observation on a noticeable number of instances.

Specifications induce much smaller game arenas due to the on-the-fly automata product.

Max-Observation has to always work on the full arena, but can generally realise more specifications and needs fewer calls.

The target sets also become complex. E.g., in some timeout cases, the target required satisfying 16 out of 20 objectives, resulting in $\binom{20}{16} = 4{,}845$ possible combinations.

Max-Guarantee is not always faster.



Cost per Iteration: Basic Max-Observation vs. Max-Guarantee

Max-Guarantee just needs a **linear** number of iterations, but it also makes each iteration more expensive.